

543. 二叉树的直径

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 二叉树
- **Companies:** (Not Available)

给你一棵二叉树的根节点 `root`，返回树的 **直径**。

二叉树的 **直径**是指树中任意两个节点之间最长路径的 **长度**。这条路径可能经过也可能不经过根节点 `root`。

两节点之间路径的 **长度**由它们之间边数表示。

示例 1:

输入：root = [1,2,3,4,5]

输出：3

解释：3，取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

示例 2:

输入：root = [1,2]

输出：1

提示:

- 树中节点数目在范围 $[1, 10^4]$ 内
- $-100 \leq \text{Node.val} \leq 100$

Discussion | Solution

解决方法

1. 问题理解

目标: 找到二叉树中任意两个节点之间最长路径的长度 (边数)。**直径定义:** 树中任意两点间最长路径的长度。**关键点:** - 路径长度是**边数**，不是节点数。- 最长路径**可能不经过根节点**。

2. 思路分析

递归 (深度优先搜索 - DFS) 对于任意一个节点 `node`: - 经过 `node` 的最长路径长度 = `node` 的左子树深度 + `node` 的右子树深度。这里的“深度”指的是从 `node` 到其子树最远叶子节点的**边数** (即节点数减 1)。- 树的直径 = 所有节点中, “经过该节点的最长路径长度”的最大值。

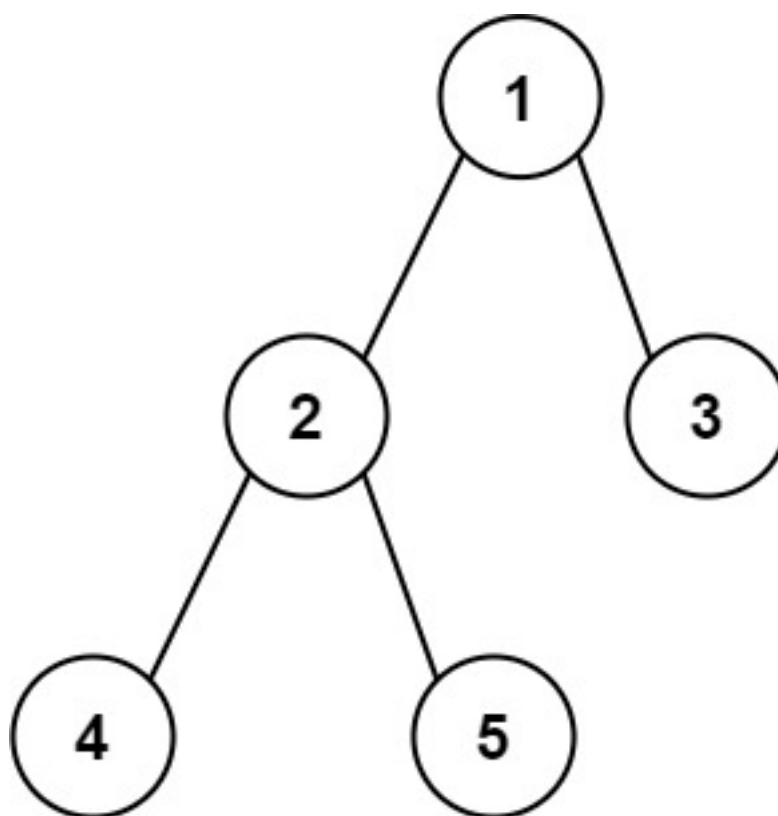


Figure 1: img

我们可以使用 DFS 来计算每个节点的深度，并在计算深度的同时更新全局的直径最大值。

1. 定义递归函数 `depth(node)`:

- **功能:** 计算以 `node` 为根的子树的深度 (边数)，并在过程中更新全局最大直径 `max_diameter`。
- **基本情况 (Base Case):** 如果 `node` 为 `null`，其深度为 `-1` (因为深度是边数，空节点没有边)。
- **递归步骤 (Recursive Step):**
 - 递归计算左子树的深度: `left_depth = depth(node.left)`。
 - 递归计算右子树的深度: `right_depth = depth(node.right)`。
 - **计算经过当前节点 `node` 的最长路径长度(直径候选值):** `current_diameter = left_depth + right_depth + 2`。 (因为 `left_depth` 和 `right_depth` 是边数，从左最深叶子到 `node` 是 `left_depth + 1` 条边，从右最深叶子到 `node` 是 `right_depth + 1` 条边，合起来是 `left_depth + right_depth + 2` 条边)。
 - **更新全局最大直径:** `max_diameter = max(max_diameter, current_diameter)`。
 - **返回当前节点的深度:** 当前节点 `node` 的深度等于其左右子树深度的较大值加 1 (加的是 `node` 到子节点的边)。 `return max(left_depth, right_depth) + 1`。

2. 主函数调用:

- 初始化全局最大直径 `max_diameter = 0`。
- 调用 `depth(root)` 开始递归计算。
- 最终返回 `max_diameter`。

注意: 递归函数 `depth` 的返回值是节点的深度 (用于给父节点计算)，而直径是在递归过程中通过比较 `left_depth + right_depth + 2` 来更新的。

执行流程示例 ([1,2,3,4,5]):

1. `max_diameter = 0`
2. `depth(1)`
3. `-> depth(2)`
4. `-> depth(4) -> depth(null)=-1, depth(null)=-1. current_diameter = -1 + -1 + 2 = 0. max_diameter=0. 返回 max(-1,-1)+1 = 0.`
5. `-> depth(5) -> depth(null)=-1, depth(null)=-1. current_diameter = -1 + -1 + 2 = 0. max_diameter=0. 返回 max(-1,-1)+1 = 0.`
6. `-> 节点 2: left_depth=0, right_depth=0. current_diameter = 0 + 0 + 2 = 2. max_diameter=2. 返回 max(0,0)+1 = 1.`
7. `-> depth(3) -> depth(null)=-1, depth(null)=-1. current_diameter = -1 + -1 + 2 = 0. max_diameter=2. 返回 max(-1,-1)+1 = 0.`
8. `节点 1: left_depth=1, right_depth=0. current_diameter = 1 + 0 + 2 = 3. max_diameter=3. 返回 max(1,0)+1 = 2.`
9. 最终返回 `max_diameter = 3`。

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        """
        计算二叉树的直径
        """
        self.max_diameter = 0 # 使用实例变量或闭包来存储全局最大直径

    def depth(node: Optional[TreeNode]) -> int:
        """
        计算以 node 为根的子树的深度 (边数), 并更新最大直径
        返回: 以 node 为根的子树的深度 (边数)
        """
        # 基本情况: 空节点深度为 -1 (方便计算边数)
        if not node:
            return -1

        # 递归计算左右子树的深度
        left_depth = depth(node.left)
        right_depth = depth(node.right)

        # 更新最大直径:
        # 经过当前 node 的最长路径长度 = 左深度 + 右深度 + 2 (两条连接边)
        # 注意 depth 返回的是边数, 所以直接 left_depth + right_depth + 2
        current_diameter = left_depth + right_depth + 2
        self.max_diameter = max(self.max_diameter, current_diameter)

        # 返回当前节点的深度 (边数) = 左右子树深度的较大值 + 1 (连接父节点的边)
        return max(left_depth, right_depth) + 1
```

```

# 从根节点开始计算深度，这个过程会更新 self.max_diameter
depth(root)

# 返回最终记录的最大直径
return self.max_diameter

```

4. 复杂度分析

- **时间复杂度：** $O(N)$ ，其中 N 是树中的节点数。每个节点被访问一次。
- **空间复杂度：** $O(H)$ ，其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下（链状树）为 $O(N)$ ，平均情况下（平衡树）为 $O(\log N)$ 。

5. 如何在面试中讲解

1. 明确直径定义：

- “二叉树的直径是任意两个节点间最长路径的长度（边数）。这条路径可能经过根节点，也可能不经过。”

2. 分析路径与节点关系：

- “对于任何一个节点，穿过它的最长路径是由『它的左子树深度』+『它的右子树深度』+ 2（连接左右子树的边）构成的。”（这里的深度指边数）
- “树的直径就是所有节点『穿过它的最长路径』中的最大值。”

3. 递归思路（DFS）：

- “我们可以设计一个递归函数，它做两件事：1. 计算当前节点的深度（边数）；2. 在计算过程中，顺便计算穿过当前节点的最长路径，并更新全局的最大直径。”
- “递归函数 `depth(node)`：
 - Base Case: `node` 为空，返回 -1（边数）。
 - 递归计算 `left_depth = depth(node.left)` 和 `right_depth = depth(node.right)`。
 - 计算穿过 `node` 的直径: `current_diameter = left_depth + right_depth + 2`。
 - 更新全局 `max_diameter = max(max_diameter, current_diameter)`。
 - 返回当前节点深度: `max(left_depth, right_depth) + 1`。”
- “主函数初始化 `max_diameter = 0`，调用 `depth(root)`，最后返回 `max_diameter`。”

4. 复杂度分析：

- “时间复杂度 $O(N)$ ，因为每个节点访问一次。”
- “空间复杂度 $O(H)$ ，递归栈的深度。”

5. 关键点与易错点：

- “注意区分节点深度（通常指节点数）和题目要求的路径长度（边数）。这里用边数定义深度更方便计算直径。”
- “递归函数的返回值是深度，用于向上传递信息；而直径是在递归过程中通过比较得到的全局变量。”
- “最长路径不一定经过根节点，所以需要在每个节点都计算穿过它的路径长度并取最大值。”